

# CO2 AT3

**Create a secure architecture for a food delivery platform focusing on customer privacy, secure payments, and order integrity.**

Name: S EZHILARASI

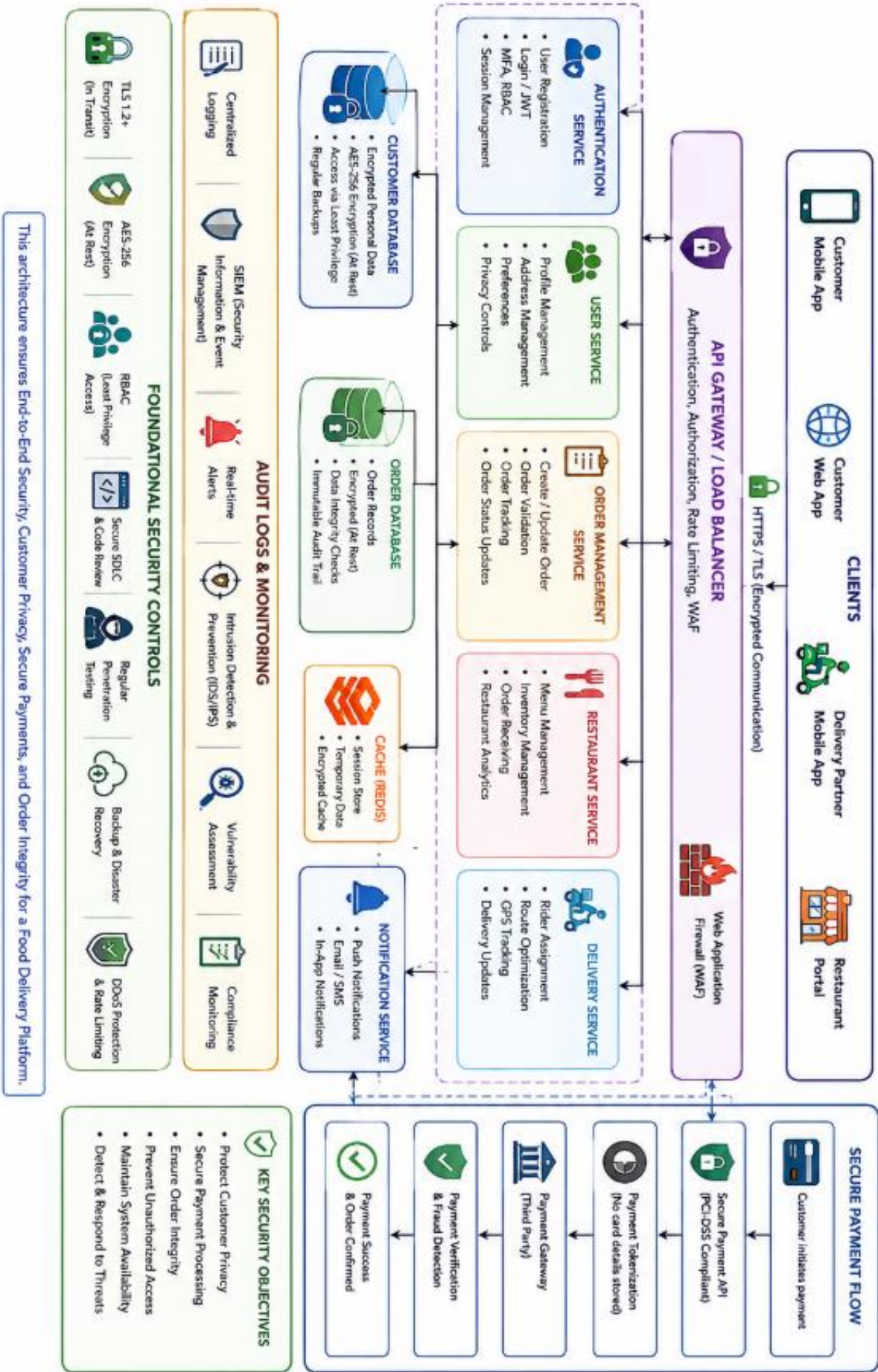
Register Number: 192521187

Course Code: CSA5803

Course Name: DevSecOps Engineering and Security Applications

# SECURE ARCHITECTURE FOR FOOD DELIVERY PLATFORM

Customer Privacy • Secure Payments • Order Integrity



## **1. Introduction**

A secure architecture is essential for food delivery platforms because they process sensitive customer information, online payments, and real-time order data. The architecture ensures confidentiality, integrity, and availability of data while protecting users from cyber threats such as unauthorized access, payment fraud, and data breaches. This design focuses on customer privacy, secure payment processing, and maintaining order integrity through multiple security layers.

## **2. Objectives**

- Protect customer personal information using strong encryption.
- Secure online payment transactions through trusted payment gateways.
- Ensure accurate and tamper-proof order processing.
- Prevent unauthorized access with authentication and authorization mechanisms.
- Monitor the system continuously to detect security threats.
- Maintain high availability and reliability of the platform.

## **3. System Components**

### **3.1 Client Layer**

The client layer consists of all users who interact with the platform:

- Customer Mobile Application
- Customer Web Application
- Delivery Partner Mobile Application
- Restaurant Portal

All communication between clients and the server occurs through HTTPS using TLS encryption to ensure secure data transmission.

### **3.2 API Gateway / Load Balancer**

The API Gateway acts as the central entry point for all incoming requests. It performs authentication, authorization, load balancing, rate limiting, request validation, and Web Application Firewall (WAF) protection. This layer blocks malicious traffic and efficiently distributes requests across backend services.

### **3.3 Authentication Service**

The Authentication Service manages secure user access through user registration, login, JWT token generation, Multi-Factor Authentication (MFA), Role-Based Access Control (RBAC), and session management. User passwords are securely stored using bcrypt hashing.

### **3.4 User Service**

The User Service manages customer-related information, including profile management, address management, user preferences, privacy settings, and account updates. All sensitive customer information is encrypted before being stored.

### **3.5 Order Management Service**

The Order Management Service handles the complete lifecycle of customer orders. Its responsibilities include order creation, order validation, order tracking, status updates, order history, and order verification. Every order is assigned a unique identifier to ensure integrity and traceability.

### **3.6 Restaurant Service**

The Restaurant Service manages restaurant operations, including menu management, inventory management, order acceptance, restaurant analytics, and food availability updates. Access to this service is restricted to authorized restaurant users.

### **3.7 Delivery Service**

The Delivery Service coordinates delivery operations by assigning riders, optimizing delivery routes, tracking GPS locations, updating delivery status, calculating estimated delivery times, and confirming successful deliveries. All communications are secured using encrypted APIs.

### **3.8 Payment Service**

The Payment Service ensures secure financial transactions using PCI-DSS compliant payment APIs, payment tokenization, third-party payment gateway integration, payment verification, fraud detection, and transaction confirmation. Customer card information is never stored within the application database.

## **4. Database Layer**

### **Customer Database**

The Customer Database stores personal information, login credentials, addresses, and user preferences. Security measures include AES-256 encryption, least-privilege access control, and regular database backups.

### **Order Database**

The Order Database stores order records, payment status, delivery details, and order history. It maintains encryption at rest, data integrity verification, and immutable audit trails for secure record management.

### **Cache (Redis)**

Redis is used for session storage, frequently accessed data, and temporary order information. Cached data is encrypted to prevent unauthorized access.

## **5. Notification Service**

The Notification Service sends push notifications, emails, SMS alerts, and order status updates. It ensures that notifications are securely delivered without exposing sensitive customer information.

## **6. Audit Logs and Monitoring**

Every significant system activity is recorded for security and compliance purposes. This includes centralized logging, Security Information and Event Management (SIEM), real-time alerts, Intrusion Detection and Prevention Systems (IDS/IPS), vulnerability assessments, and compliance monitoring. These capabilities help detect attacks and support incident investigations.

## **7. Foundational Security Controls**

The architecture follows industry-standard security practices, including TLS 1.2/1.3 encryption for data in transit, AES-256 encryption for data at rest, Role-Based Access Control (RBAC), Secure Software Development Life Cycle (SSDLC), secure code reviews, regular penetration testing, backup and disaster recovery mechanisms, DDoS protection, and API rate limiting. These controls provide multiple layers of defense against cyber threats.

## **8. Secure Payment Flow**

The payment process begins when the customer initiates a payment. The request is sent to a secure payment API, where sensitive payment details are tokenized before being forwarded to a PCI-DSS compliant payment gateway. The gateway verifies the payment and performs fraud detection checks. Once the transaction is approved, payment confirmation is returned, the order is confirmed, and the customer receives a notification.

## **9. Data Flow**

The customer accesses the application through a secure HTTPS connection. The API Gateway authenticates and validates the request before forwarding it to the Authentication Service. After successful verification, a JWT token is issued to the user. The customer places an order, which is processed by the Order Management Service. The Payment Service securely completes the transaction before forwarding the confirmed order to the Restaurant Service. The Delivery Service assigns a delivery partner and tracks the order in real time. Finally, the Notification Service updates the customer, while all system activities are securely recorded in audit logs.

## **10. Key Security Objectives**

- Protect customer privacy.
- Ensure secure payment processing.
- Maintain order integrity.
- Prevent unauthorized access.
- Ensure high system availability.
- Detect and respond to cyber threats.
- Comply with industry security standards.
- Build customer trust through secure operations.

## **11. Advantages**

- Provides strong protection for customer data.
- Ensures secure and reliable payment processing.
- Improves system scalability and availability.
- Reduces the risk of fraud and cyberattacks.
- Supports compliance with security regulations.
- Enhances trust among customers, restaurants, and delivery partners.

- Enables easy integration of future security enhancements.
- Simplifies monitoring, auditing, and incident response.

## **12. Conclusion**

The proposed secure architecture provides a comprehensive framework for protecting a food delivery platform against modern cybersecurity threats. By integrating secure authentication, encrypted communication, PCI-DSS compliant payment processing, role-based access control, audit logging, and continuous monitoring, the platform ensures customer privacy, secure payments, and order integrity. This layered security approach improves reliability, supports regulatory compliance, and establishes a secure, scalable, and trustworthy environment for customers, restaurants, and delivery partners.